Fundamentals of Backend Engineering

for Data Scientist

2025

Lê Thiện Hòa

Nội dung

- Web API Fundamentals, URL with path parameters & query parameters
- OSI Models with 7 layers, TCP & UDP, How HTTP & HTTPS work
- Restful API, Building APIs with Flask & FastAPI
- WSGI & ASGI (Async & Await), Werkzeug & Gunicorn & Uvicorn Server
- Encoding (Base64), Encryption (Symmetric AES & Asymmetric RSA), Hashing (SHA256, bcrypt)
- API Security: Authentication & Authorization (Stateful API & API Key & Basic Auth & JWT & OAuth2)
- RPC (Thrift / gRPC): Connection Pooling, Transport Layer, Multiplexing, Streaming
- Websocket, Webhook and Celery task

Web API

API = Application Programming Interface

Cho phép **ứng dụng giao tiếp** với nhau qua HTTP

Thường dùng để:

- Truy vấn dữ liệu (GET)
- Gửi dữ liệu (POST)
- Cập nhật (PUT/PATCH)
- Xóa (DELETE)

Là “cầu nối” giữa model/data và client

Web API & Database CRUD Operations

Liên hệ với Database CRUD

Database Operation	HTTP Method (API)	Mô tả
Create	POST	Thêm bản ghi mới
Read	GET	Lấy dữ liệu hiện có
Update	PUT / PATCH	Chỉnh sửa bản ghi
Delete	DELETE	Xóa bản ghi

Giúp client **thao tác dữ liệu qua HTTP** mà không cần truy cập trực tiếp vào database

URL

Cấu trúc tổng quát:

- scheme://domain:port/path?query

Ví dụ:

- https://api.example.com:443/users/123/orders?status=delivered&page=2

Thành phần	Ví dụ	Ý nghĩa
Scheme / Protocol	`https://`	Giao thức truyền (HTTP / HTTPS)
Domain	`api.example.com`	Tên máy chủ (server API)
Port (tùy chọn)	`:443`	Cổng kết nối (443 cho HTTPS, 80 cho HTTP)
Path	`/users/123/orders`	Đường dẫn đến tài nguyên cụ thể
Query	`?status=delivered&page=2`	Tham số lọc / tùy chọn

- **Domain + Path** → xác định tài nguyên (resource)
- **Query** → xác định cách truy xuất (filter, sort, paginate)
- **HTTPS** → giúp mã hóa toàn bộ URL + body

URL Ports

Port	Giao thức (Protocol)	Mục đích sử dụng
20 / 21	FTP (File Transfer Protocol)	Truyền tải file giữa client ↔ server
22	SSH (Secure Shell)	Truy cập máy chủ từ xa an toàn
23	Telnet	Truy cập máy chủ từ xa (không mã hóa — lỗi thời)
25	SMTP	Gửi email (Mail Transfer)
53	DNS (Domain Name System)	Phân giải tên miền → IP
67 / 68	DHCP	Cấp phát IP tự động
80	HTTP	Giao tiếp web không mã hóa
443	HTTPS	Giao tiếp web **mã hóa (bảo mật)**
110	POP3	Nhận email từ server (Mail client)
143	IMAP	Nhận email có đồng bộ nhiều thiết bị

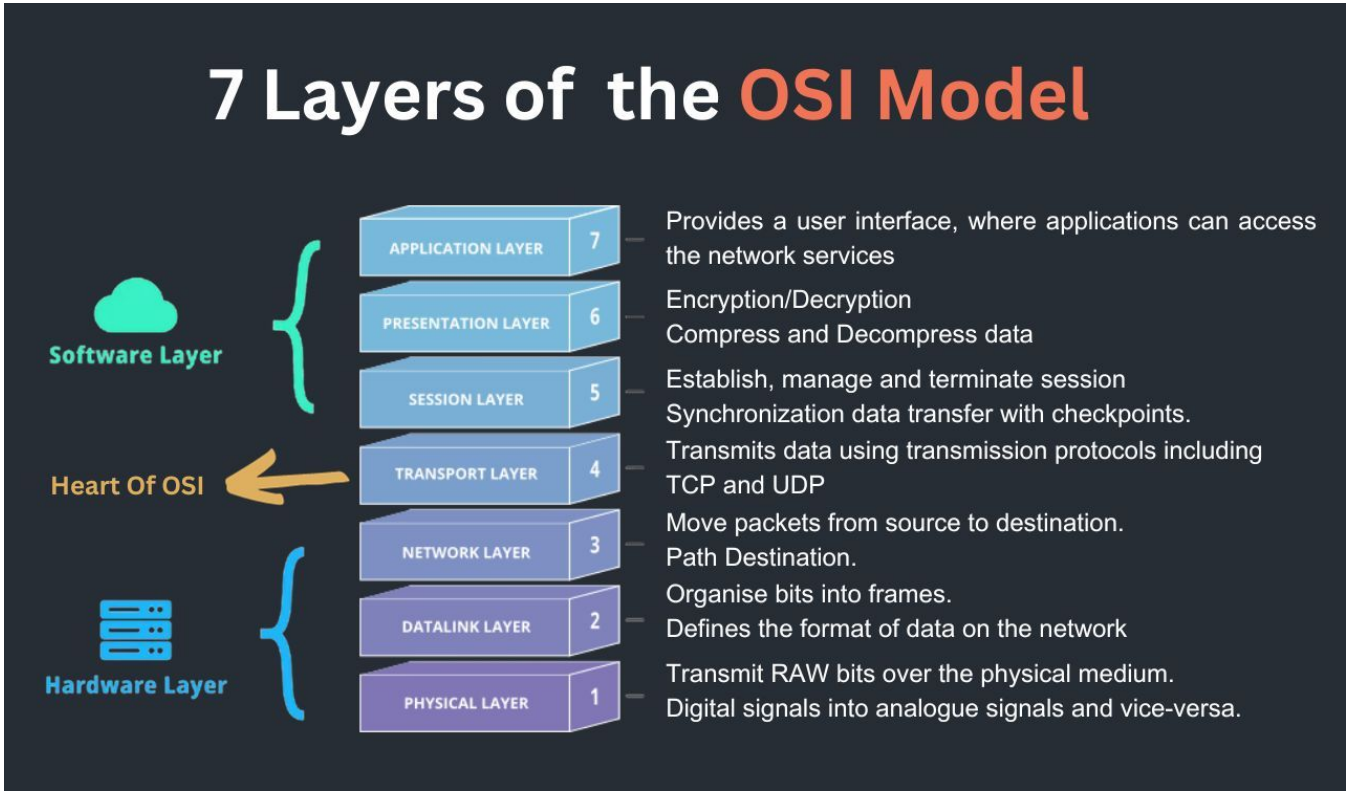
URL Ports

Port	Giao thức (Protocol)	Mục đích sử dụng
3306	MySQL	Cổng mặc định của MySQL Database
5432	PostgreSQL	Cổng mặc định của PostgreSQL
6379	Redis	In-memory cache / message broker
27017	MongoDB	Kết nối tới MongoDB database
8080	HTTP (alternative)	Thường dùng cho web dev / proxy / API test
8000 / 8001	Dev server (Python / Node / FastAPI / Django)	Cổng mặc định khi chạy local server
9092	Kafka	Message broker (stream processing)
15672	RabbitMQ Management UI	Dashboard quản lý RabbitMQ
5000	Flask API / Local service	Mặc định của Flask hoặc REST API local
6379	Redis	Cache và hàng đợi task (Celery, v.v.)

URL Ports

- **80** → **HTTP**, **443** → **HTTPS** là hai cổng web quan trọng nhất.
- **Database ports** (3306, 5432, 27017) thường chỉ mở trong nội bộ.
- **SSH (22)** luôn cần bảo mật kỹ vì là cổng truy cập máy chủ.
- **Dev ports (8000, 8080, 5000)** chỉ dùng khi chạy local, không nên mở public.

OSI Model


7 Layers of the OSI Model
Software Layer
Heart Of OSI
Hardware Layer
APPLICATION LAYER
7
Provides a user interface, where applications can access the network services
PRESENTATION LAYER
6
Encryption/Decryption
Compress and Decompress data
SESSION LAYER
5
Establish, manage and terminate session
Synchronization data transfer with checkpoints.
TRANSPORT LAYER
4
Transmits data using transmission protocols including TCP and UDP
NETWORK LAYER
3
Move packets from source to destination.
Path Destination.
DATALINK LAYER
2
Organise bits into frames.
Defines the format of data on the network
PHYSICAL LAYER
1
Transmit RAW bits over the physical medium.
Digital signals into analogue signals and vice-versa.

OSI Model

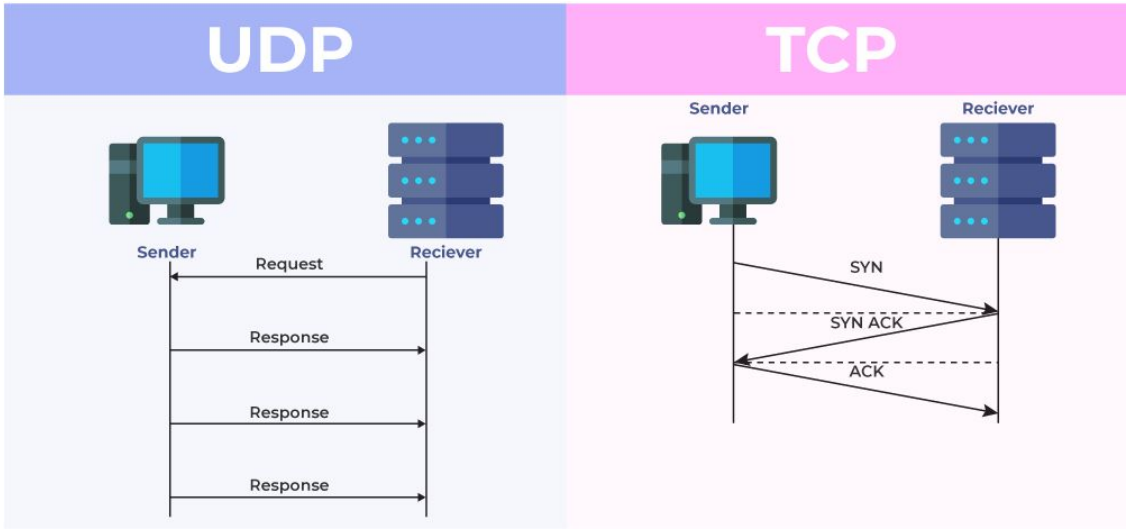
Giao tiếp giữa ứng dụng và người dùng

Định tuyến và truyền tin qua internet

Truyền dữ liệu nội bộ qua phần cứng

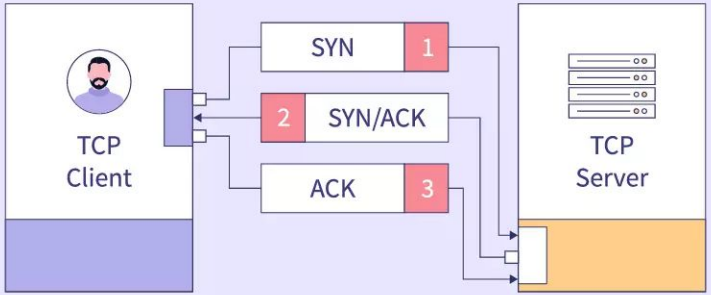
#	Tên lớp (Tên tiếng Anh)	Nhiệm vụ chính	Đơn vị dữ liệu	Ví dụ cụ thể
7 **Application**	Lớp Ứng dụng	Giao tiếp trực tiếp với người dùng / ứng dụng	**Data**	HTTP, FTP, SMTP, DNS
6 **Presentation**	Lớp Trình bày	Mã hóa, nén, chuyển định dạng dữ liệu	**Data**	SSL/TLS, JSON, JPEG, ASCII
5 **Session**	Lớp Phiên	Thiết lập, quản lý và kết thúc kết nối	**Data**	Session ID, handshake
4 **Transport**	Lớp Vận chuyển	Truyền dữ liệu đáng tin cậy, chia gói	**Segment**	TCP, UDP, Port 80/443
3 **Network**	Lớp Mạng	Định tuyến (routing), địa chỉ IP	**Packet**	IP, ICMP, Router
2 **Data Link**	Lớp Liên kết dữ liệu	Đóng gói frame, kiểm tra lỗi vật lý	**Frame**	Ethernet, MAC Address
1 **Physical**	Lớp Vật lý	Truyền bit qua dây / sóng	**Bits**	Cáp quang, dây đồng, Wi-Fi

OSI Model - TCP vs UDP



2 giao thức truyền dữ liệu chính trong tầng **Transport** (tầng 4 của mô hình OSI) — chịu trách nhiệm truyền dữ liệu từ ứng dụng này sang ứng dụng khác giữa hai thiết bị qua mạng

OSI Model - TCP

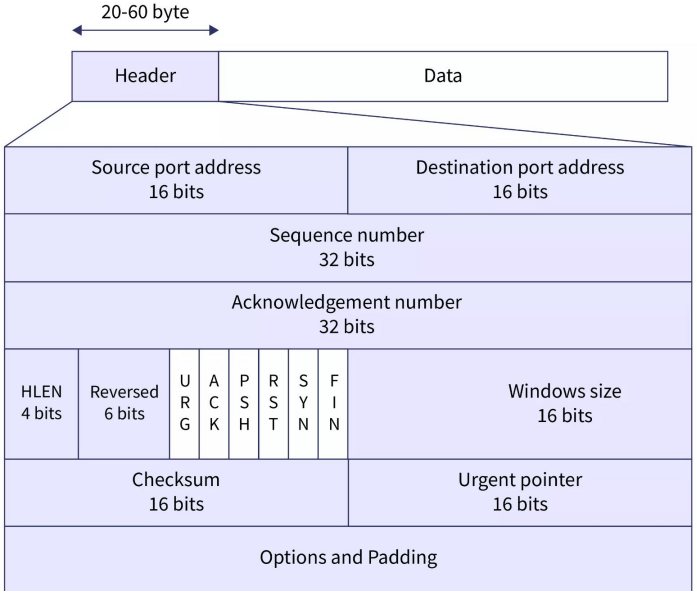


TCP hoạt động theo tiến trình bắt tay 3 bước (**3 way handshake**):

- Máy khách gửi cho máy chủ một gói **SYN** — một yêu cầu kết nối từ port nguồn của nó đến port đích đến của máy chủ.
- Máy chủ phản hồi bằng gói **SYN/ACK**, xác nhận việc nhận được yêu cầu kết nối.
- Máy khách nhận gói **SYN/ACK** và trả lời bằng gói **ACK** của chính nó.

OSI Model - TCP

Cấu trúc của TCP Header



Đặc điểm:

- Là giao thức có kết nối (connection-oriented).
- Đảm bảo dữ liệu đến đúng nơi, đúng thứ tự và không bị mất.
- Kiểm tra lỗi và đảm bảo thứ tự gói tin (sequence number).
- Gửi lại gói bị mất, kiểm soát tắc nghẽn (congestion control).

Ưu điểm:

- Đảm bảo độ tin cậy cao.
- Thứ tự gói tin được duy trì chính xác.

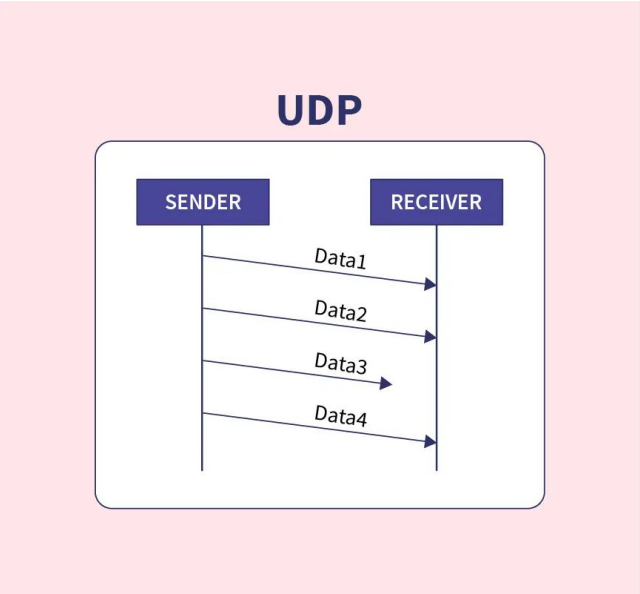
Nhược điểm:

- Tốc độ chậm hơn do nhiều bước xác nhận và kiểm soát lỗi.

Ứng dụng phổ biến:

- Web (HTTP/HTTPS)
- Email (SMTP, IMAP, POP3)
- File transfer (FTP, SFTP)

OSI Model - UDP



Đặc điểm:

- Là giao thức không kết nối (connectionless).
- Không đảm bảo dữ liệu đến đủ, đúng thứ tự hay không bị mất.

Cơ chế:

- Gửi gói tin (“datagram”) trực tiếp, không cần bắt tay 3 bước.
- Không có cơ chế kiểm soát lỗi hoặc truyền lại.

Ưu điểm:

- Tốc độ rất nhanh, độ trễ thấp.
- Tối ưu cho các ứng dụng real-time (âm thanh, video, game online).

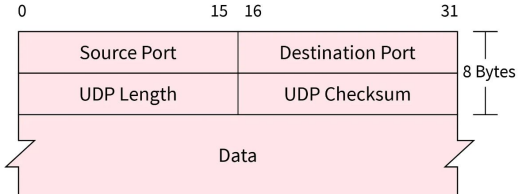
Nhược điểm:

- Không đảm bảo độ tin cậy hoặc thứ tự.

Ứng dụng phổ biến:

- Streaming (video/audio live)
- Voice over IP (VoIP)
- Online gaming
- DNS (Domain Name System)

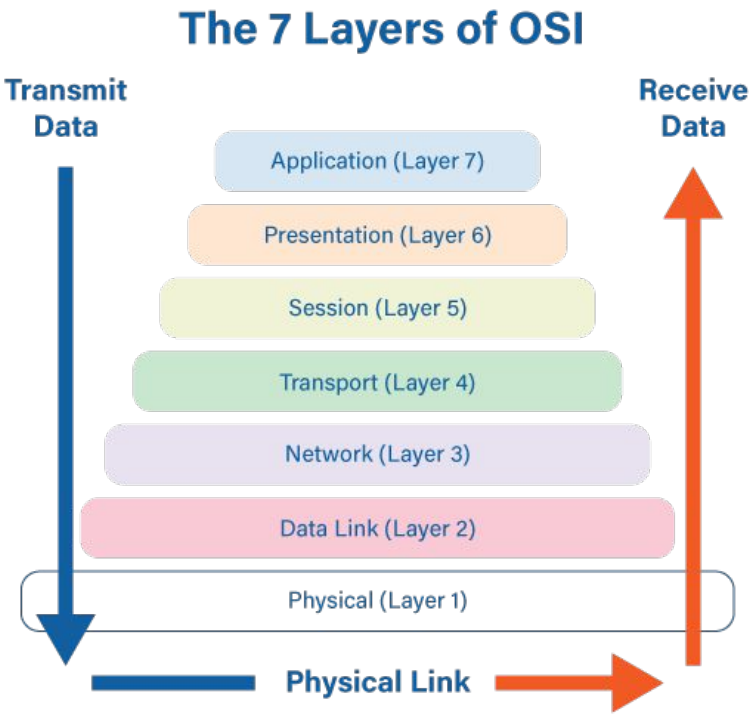
Cấu trúc của UDP Header



OSI Model - TCP vs UDP

Tính năng	TCP	UDP
Hướng kết nối	Có	Không
Tính đáng tin cậy	Có	Không
Tính toàn vẹn	Có	Không
Tính có thể tái tạo	Có	Không
Kiểm soát luồng	Có	Không
Tốc độ	Chậm hơn	Nhanh hơn
Sử dụng	Các ứng dụng yêu cầu tính đáng tin cậy cao, chẳng hạn như FTP, SSH và HTTP	Các ứng dụng không yêu cầu tính đáng tin cậy cao, chẳng hạn như phát trực tiếp video và âm thanh

OSI Model - Send & Receive Messages



Khi client gửi request:

> Application → Presentation → Session → Transport → Network → Data Link → Physical → Dữ liệu được đóng gói dần (Encapsulation)

Khi server nhận:

> Physical → Data Link → Network → Transport → Session → Presentation → Application → Dữ liệu được giải mã dần (Decapsulation)

OSI Model - HTTP Client send request (Encapsulation)

Mỗi lớp OSI sẽ **“đóng gói” (encapsulate)** dữ liệu thêm một lớp header riêng trước khi gửi xuống lớp dưới.

Ví dụ: Client gửi request HTTP đến server.

Bước	Lớp	Vai trò
7	**Application**	Sinh ra dữ liệu (HTTP Request)	`"GET /index.html HTTP/1.1"`
6	**Presentation**	Mã hóa / nén (nếu có TLS)	`Encrypted(HTTP data)`
5	**Session**	Quản lý session / handshake	`Session info + Encrypted(HTTP)`
4	**Transport**	Thêm **TCP header** (port nguồn & đích)	`TCP Header + Session + Data`
3	**Network**	Thêm **IP header** (IP nguồn & đích)	`IP Header + TCP + Data`
2	**Data Link**	Thêm **MAC header**	`MAC Header + IP + TCP + Data`
1	**Physical**	Chuyển thành bit điện / sóng	`01010101...` truyền qua dây hoặc Wi-Fi

OSI Model - HTTP Server receive request (Decapsulation)

Server sẽ **làm ngược lại** — "mở từng lớp bọc" để lấy dữ liệu gốc ra.

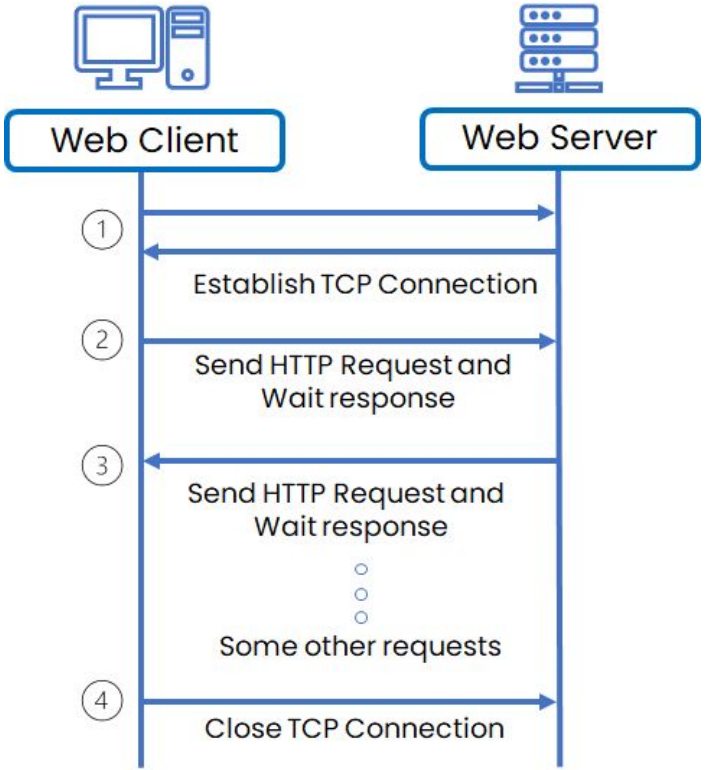
Bước	Lớp	Vai trò
1	**Physical**	Nhận tín hiệu	Chuyển bit thành frame
2	**Data Link**	Đọc MAC, kiểm tra lỗi	Bỏ MAC header
3	**Network**	Đọc IP header	Kiểm tra IP đích
4	**Transport**	Đọc TCP/UDP header	Kiểm tra port (ví dụ 80, 443)
5	**Session**	Khôi phục phiên	Bỏ header session
6	**Presentation**	Giải mã / giải nén	Giải mã TLS
7	**Application**	Đọc nội dung HTTP	Trả dữ liệu cho ứng dụng web

OSI Model - Send & Receive Messages

Giống như khi ta gửi một bức thư:

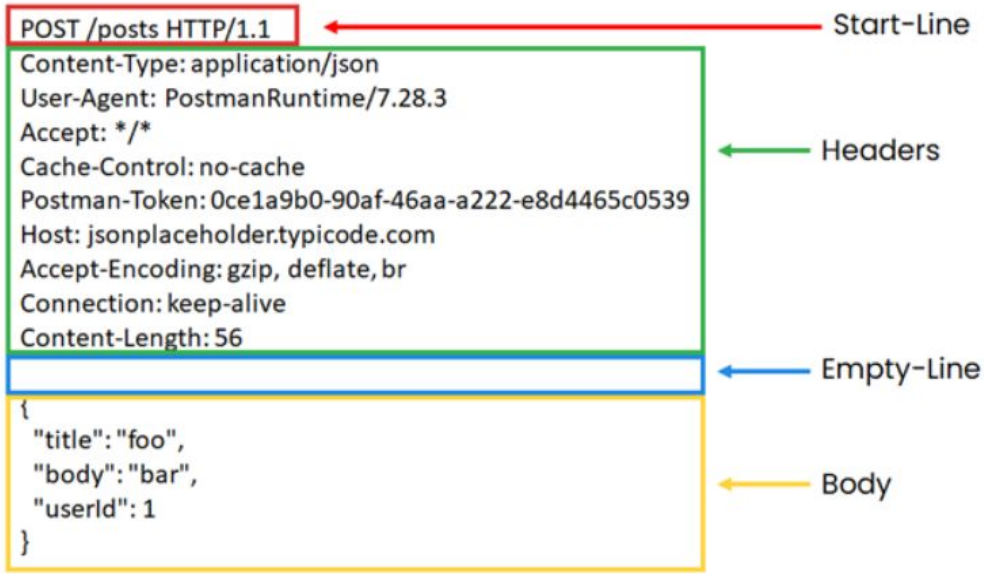
- **Application** = nội dung thư
- **Presentation** = mã hóa thư
- **Session** = gói thư vào phong bì
- **Transport** = ghi địa chỉ người gửi / người nhận
- **Network** = chọn đường đi qua bưu điện
- **Data Link** = đưa lên xe bưu điện
- **Physical** = vận chuyển thật sự

URL - How HTTP works



- Thiết bị client thiết lập kết nối TCP với server (thường qua cổng 80).
- Client gửi request theo cú pháp HTTP và chờ phản hồi từ server.
- Server xử lý request và gửi lại response kèm mã trạng thái và dữ liệu (nếu có).
- Client đóng kết nối TCP; trước khi đóng, có thể lặp lại bước gửi và nhận nhiều lần nếu cần (khi server/client chủ động ngắt kết nối hoặc timeout)

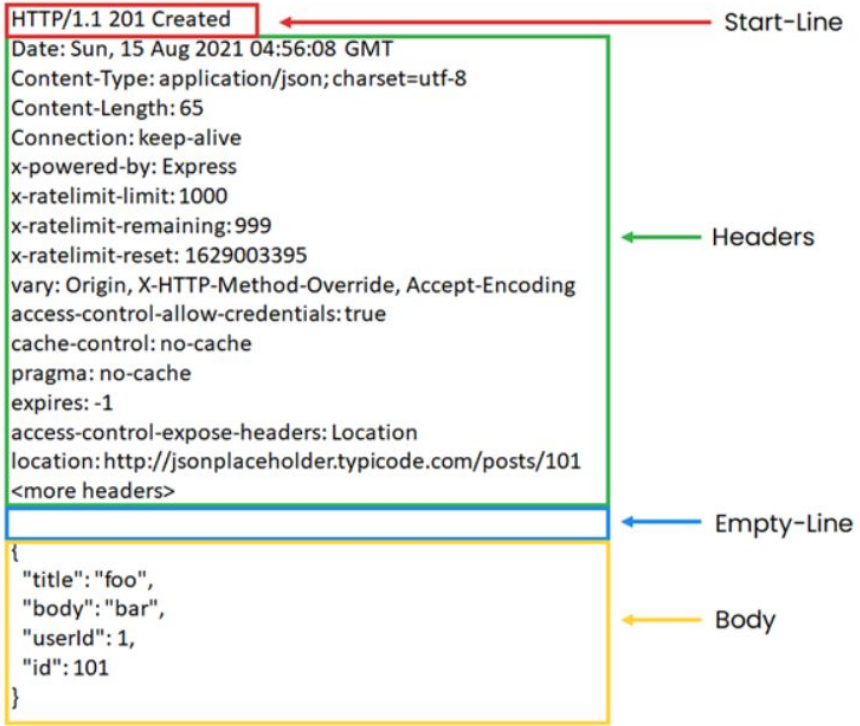
URL - HTTP Request Messages



HTTP Message Request

- **Start-Line:** Mở đầu request/response, chứa loại, trạng thái và phiên bản HTTP.
- **Headers:** Thông tin bổ sung trao đổi giữa client và server (chứa cache vì http bản chất là stateless)
- **Empty-Line:** Đánh dấu kết thúc phần header.
- **Body:** Chứa dữ liệu (payload) gửi hoặc nhận.

URL - HTTP Response Messages



HTTP Response Request

- **HTTP** được tạo ra để hoạt động như giao thức ứng dụng trên **TCP**, nhưng có thể dùng **SSL/TLS** để trở thành **HTTPS**.
- **HTTP** là giao thức **stateless**, không lưu trạng thái; quản lý phiên bằng **Cookies** hoặc **Tokens** trong Header.
- Hiện nay, **HTTP** gắn liền với khái niệm **REST** (REST-API hoặc RESTful API).
- **Khuyến nghị**: Nên dùng **HTTPS** để đảm bảo an toàn khi truyền tải nội dung web.

HTTP - GET

- Dùng để **lấy dữ liệu** từ server
- Dữ liệu truyền qua **URL (query string)**

  → Ví dụ: /users?name=hoa&id=123
- **Không có body**
- Dễ cache, bookmark
- **Không bảo mật** vì dữ liệu hiện trên URL và log server

HTTP - POST

- Dùng để **gửi hoặc tạo dữ liệu mới**
- Dữ liệu gửi trong **body của request**

  Ví dụ:

```
POST /users
Content-Type: application/json

{ "name": "Hoa", "email": "hoa@example.com" }
```

- **Bảo mật hơn GET**, vì dữ liệu không hiển thị trên URL (nhưng vẫn cần HTTPS để mã hóa)
- Không được cache, không bookmark được

HTTP - PUT

- Dùng để **cập nhật toàn bộ** tài nguyên
- Cần gửi **toàn bộ dữ liệu mới**, kể cả phần không thay đổi
- Nếu tài nguyên chưa tồn tại, có thể tạo mới (tùy thiết kế)

Ví dụ

```
PUT /users/123
Content-Type: application/json

{ "name": "Hoa", "email": "hoa@example.com" }
```

→ Cập nhật toàn bộ thông tin user có ID = 123

HTTP - PATCH

- Dùng để **cập nhật một phần** tài nguyên
- Chỉ cần gửi **trường thay đổi**
- Hiệu quả hơn khi chỉnh sửa nhỏ

  Ví dụ

```
PATCH /users/123
Content-Type: application/json

{ "email": "new_email@example.com" }
```

→ Chỉ cập nhật trường email cho user 123

HTTP - DELETE

- Dùng để **xóa** tài nguyên

  Ví dụ:

```
DELETE /users/123
```

  → Xóa user có ID = 123

HTTP - Common Error Code

🟢 2xx – Success

Mã	Ý nghĩa	Mô tả
200 OK	Thành công	Request xử lý bình thường, trả dữ liệu về
201 Created	Tạo mới thành công	Dữ liệu đã được thêm (POST thành công)

🔵 3xx – Redirection

Mã	Ý nghĩa	Mô tả
301 Moved Permanently	Chuyển hướng vĩnh viễn	URL đã thay đổi
302 Found	Chuyển hướng tạm thời	Tạm thời sang URL khác

HTTP - Common Error Code

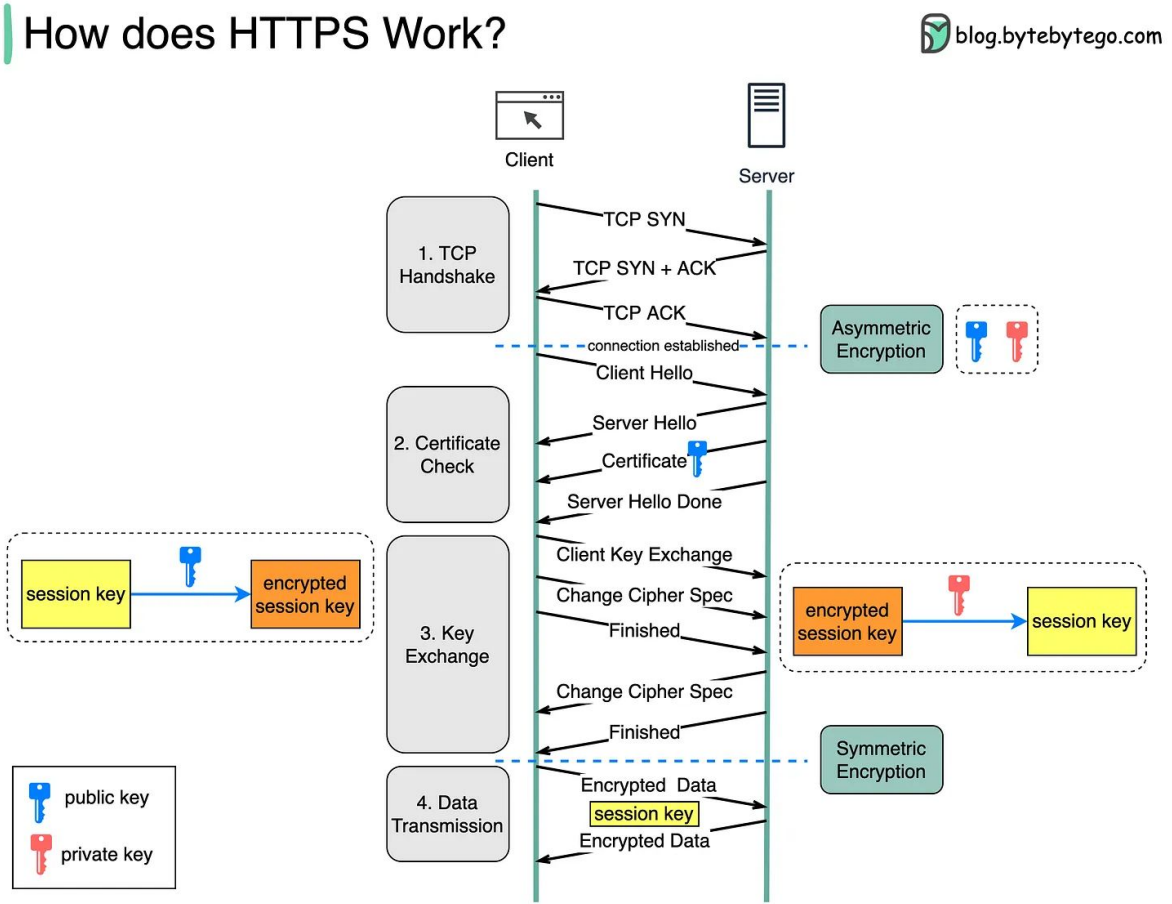
4xx – Client Error

Mã	Ý nghĩa	Mô tả
400 Bad Request	Request sai cú pháp	Thiếu field, JSON lỗi
401 Unauthorized	Chưa xác thực	Thiếu/invalid token
403 Forbidden	Không có quyền	Có token nhưng không được phép
404 Not Found	Không tìm thấy tài nguyên	Sai URL hoặc ID không tồn tại
409 Conflict	Xung đột dữ liệu	Tạo trùng, trạng thái không hợp lệ

5xx – Server Error

Mã	Ý nghĩa	Mô tả
500 Internal Server Error	Lỗi phía server	Code lỗi, exception
502 Bad Gateway	Gateway nhận lỗi từ upstream	Lỗi giữa các service
503 Service Unavailable	Server quá tải / bảo trì	Thường dùng khi deploy
504 Gateway Timeout	Timeout giữa các service	API gọi lâu không phản hồi

URL - How HTTPS works



Bước 1: Client (trình duyệt) và server thiết lập kết nối **TCP**.

Bước 2: Client gửi **"client hello"** với danh sách thuật toán mã hóa và phiên bản **TLS** hỗ trợ; server phản hồi **"server hello"** và gửi **chứng chỉ SSL** chứa khóa công khai. Client xác thực chứng chỉ.

Bước 3: Client tạo **session key**, mã hóa bằng khóa công khai và gửi cho server; server giải mã bằng **khóa riêng**.

Bước 4: Cả hai cùng chia sẻ **session key**, trao đổi dữ liệu an toàn qua kênh mã hóa đối xứng.

URL - HTTP vs. HTTPS

	HTTP	HTTPS
Port mặc định	80	443
Bảo mật	❌ Không mã hóa	✅ Mã hóa dữ liệu
Hiển thị URL	`http://`	`https://`
Nguy cơ	Dễ bị nghe lén (sniffing), giả mạo (MITM)	An toàn nhờ mã hóa TLS

HTTPS hoạt động như thế nào

1. **Client** gửi yêu cầu đến server (`https://example.com`)
2. **Server** gửi **chứng chỉ số (SSL/TLS certificate)**
3. **Client** kiểm tra chứng chỉ có hợp lệ không (CA xác nhận)
4. Hai bên thực hiện **TLS Handshake** → sinh ra **khoá mã hóa tạm thời (session key)**
5. Từ đó, mọi dữ liệu **đều được mã hóa** (encrypted) khi truyền qua mạng

URL - HTTP vs. HTTPS

HTTPS:

- Thông tin (body, header, token, password, JSON data, v.v.) được **mã hóa**
- Hacker hoặc proxy trung gian **chỉ thấy dữ liệu mã hóa**, không đọc được nội dung

Vì sao **quan trọng cho API**:

- Bảo vệ **token / password / user info** khi truyền qua network
- Ngăn chặn **Man-in-the-Middle attack**
- Là **yêu cầu bắt buộc** với các hệ thống production (OAuth, JWT, Banking API...)

URL - HTTP vs. HTTPS

OSI Layer	HTTP	HTTPS	Giải thích
7 **Application**	Giao thức HTTP (HyperText Transfer Protocol)	Giao thức **HTTP chạy bên trong SSL/TLS**	Hai bên đều xử lý request/response, nhưng HTTPS thêm lớp mã hóa
6 **Presentation**	❌ Không có mã hóa	✅ **SSL/TLS mã hóa** dữ liệu trước khi gửi	Đây là **điểm khác biệt chính**: HTTPS đảm bảo tính bảo mật và toàn vẹn dữ liệu
5 **Session**	TCP session bình thường	TCP session có **bắt tay TLS** (handshake)	Xác thực chứng chỉ, trao đổi khóa
4 **Transport**	TCP (Port **80**)	TCP (Port **443**)	Dùng cùng giao thức TCP nhưng **khác port**
3 **Network**	IP	IP	Không khác biệt
2 **Data Link**	Ethernet	Ethernet	Không khác biệt
1 **Physical**	Bit truyền qua cáp / Wi-Fi	Bit truyền qua cáp / Wi-Fi	Không khác biệt

URL - HTTP vs. HTTPS

Thành phần	HTTP	HTTPS
Port mặc định	80	443
Mã hóa dữ liệu	Không	Có (SSL/TLS)
Bảo mật thông tin đăng nhập / token	Dễ bị đánh cắp	Được bảo vệ
Tốc độ	Nhanh hơn một chút	Chậm hơn chút do mã hóa
Tin cậy (Browser)	Không hiển thị "Secure"	Có biểu tượng ổ khóa 🔒

- HTTP = Gửi thẳng dữ liệu dạng text (Layer 7)
- HTTPS = Gửi qua SSL/TLS (Layer 6), rồi mới xuống Transport (Layer 4)

URL - Path parameters & Query parameters

Query Parameters:

- Dữ liệu được gửi kèm trên URL, sau dấu ?
- Thường dùng cho truy vấn (GET), lọc dữ liệu, hoặc tùy chọn nhỏ
- Giới hạn độ dài (URL max length)
- Không phù hợp để gửi dữ liệu nhạy cảm hoặc lớn

Ví dụ:

- GET /users?country=VN&active=true

URL - Path parameters & Query parameters

Path Parameters:

- Nằm **trực tiếp trong URL**
- Dùng để **xác định một tài nguyên cụ thể**
- Không thay đổi ý nghĩa endpoint
- Thường là **bắt buộc**

Ví dụ:

- /users/123
- /products/456/reviews

→ Lấy user có **ID = 123**, hoặc review của **product 456**

URL - Path parameters & Query parameters

Query Parameters

- Nằm **sau dấu ?** trong URL
- Dùng để **lọc (filter), sắp xếp, tìm kiếm, hoặc phân trang**
- Thường là **tùy chọn**

Ví dụ:

- /users?country=VN&active=true
- /products?category=phone&limit=10&page=2

→ Lọc user theo quốc gia, hoặc giới hạn danh sách sản phẩm

URL - Path parameters & Query parameters

Kết hợp cả hai

- /users/123/orders?status=delivered

→ Lấy danh sách đơn hàng của user 123, lọc theo trạng thái đã giao (delivered)

Loại	Vị trí trong URL	Mục đích chính	Ví dụ
Path Param	`/users/{id}`	Xác định tài nguyên cụ thể	`/users/123`
Query Param	`/users?key=value`	Lọc / tùy chọn / phân trang	`/users?country=VN`

Web API

Các loại API phổ biến:

- **REST API** → text/json, phổ biến nhất (one way)
- **RPC (gRPC / Thrift)** → binary, hiệu năng cao, dùng nội bộ service
- **WebSocket** → giao tiếp 2 chiều real-time
- **Webhook** → callback tự động giữa hệ thống (event driven)

Web API

Đặc điểm	REST API	WebSocket	Webhook
Kiểu giao tiếp	Request/Response (client gọi, server trả lời)	Persistent connection (2 chiều, giữ kết nối liên tục)	Event-driven (server gọi ngược về client endpoint)
Giao thức	HTTP	WS (WebSocket protocol, khởi đầu từ HTTP handshake)	HTTP
Chiều dữ liệu	1 chiều: Client → Server	2 chiều: Client ↔ Server	1 chiều: Server → Client
Dữ liệu gửi	JSON, XML,...	Bất kỳ (JSON, binary, text...)	Thường JSON (qua HTTP POST)
Dùng khi nào	Lấy dữ liệu, thao tác CRUD	Realtime data, chat, streaming	Trigger callback khi có sự kiện (payment, training done, v.v.)

Restful API - Flask with Werkzeug (dev server by default)

Simple **GET** API

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/hello', methods=['GET'])
def hello():
    return jsonify({'message': 'Hello, REST API!'})

if __name__ == '__main__':
    app.run(debug=True)
```

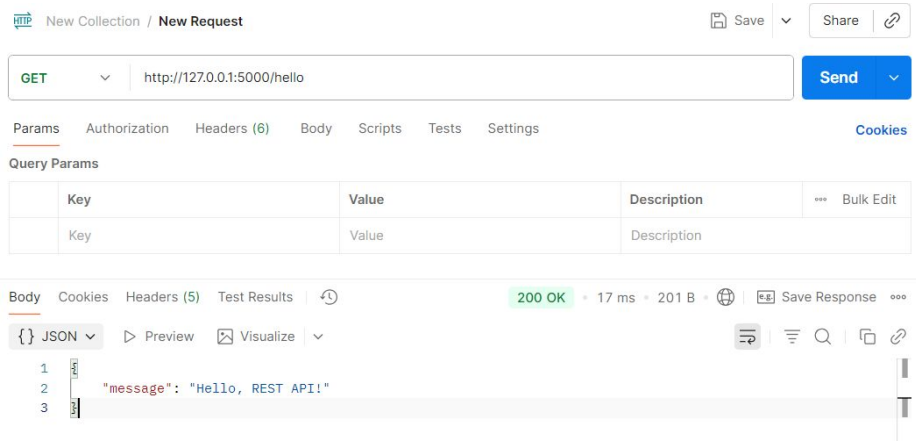
Run: python simple_api.py

Restful API - Flask

- **Browser**



- **Postman**



- **Curl**

```
(base) PS D:\H\Cole vn\Python\Buổi 13\restful_api\flask> curl http://127.0.0.1:5000/hello


                                    StatusCode        : 200

StatusDescription : OK
Content           : {
                      "message": "Hello, REST API!"
                    }

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 36
                    Content-Type: application/json
                    Date: Sat, 11 Oct 2025 05:09:10 GMT
                    Server: Werkzeug/3.1.3 Python/3.13.2

                    {
                      "message": "Hello, REST API!"
                    }...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 36],
                    [Content-Type, application/json], [Date,
                    Sat, 11 Oct 2025 05:09:10 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 36
```

Restful API - Flask

POST method

```
curl -X POST -H "Content-Type: application/json" \
-d '{"name":"Charlie"}' http://127.0.0.1:5000/users
```

Thành phần	Ý nghĩa	Ghi chú
`curl`	Command-line tool để gửi HTTP request	Dùng test API, upload/download dữ liệu
`-X POST`	Xác định HTTP method là **POST**	Các method khác: `GET`, `PUT`, `DELETE`, `PATCH`, v.v.
`-H "Content-Type: application/json"`	Thêm **header** vào request	Header này nói rằng dữ liệu gửi đi là **JSON**
`-d '{"name":"Charlie"}'`	**Data** gửi trong body của request	Tương đương `request.body` trong Flask
`http://127.0.0.1:5000/users`	**Endpoint URL** của API	127.0.0.1 = localhost (chạy trên máy bạn), cổng 5000 là mặc định của Flask

Restful API - Flask

POST method

```
curl -X POST -H "Content-Type: application/json" \
-d '{"name":"Charlie"}' http://127.0.0.1:5000/users
```

Tham số	Ý nghĩa
`-v`	Hiện chi tiết request/response (debug mode)
`-i`	Hiển thị cả header của response
`-X GET`	Gửi request kiểu GET
`-X PUT`	Cập nhật tài nguyên
`-X DELETE`	Xóa tài nguyên
`-d @file.json`	Gửi dữ liệu từ file thay vì gõ trực tiếp
`-H "Authorization: Bearer <token>"`	Thêm token để test API có bảo mật

Restful API - Flask with Gunicorn (production server)

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/hello', methods=['GET'])
def hello():
    return jsonify({'message': 'Hello, REST API!'})
```

Run: gunicorn simple_api:app --workers 4 --bind 0.0.0.0:5000

Restful API - Werkzeug vs. Gunicorn

Tiêu chí	Werkzeug Server (Flask built-in)	Gunicorn
Loại	Thư viện WSGI toolkit và dev server tích hợp trong Flask	**WSGI HTTP Server độc lập cho Python** (không thuộc Flask)
Vai trò chính	Dùng để chạy Flask app nhanh khi phát triển	**Production server** chạy mọi app WSGI (Flask, Django, FastAPI qua Uvicorn Worker)
Nguồn gốc	Thuộc Flask (dựa trên thư viện Werkzeug)	Dự án độc lập, tương thích chuẩn WSGI
Hiệu năng	Thấp – chỉ 1 process, blocking, xử lý tuần tự	Cao – hỗ trợ **multi-worker / multi-process**, **load balancing**, **graceful reload**, **logging**, **timeout**, **pre-fork model**
Cơ chế xử lý	Chạy 1 thread → các request đến cùng lúc sẽ phải đợi	Chạy nhiều worker song song → xử lý nhiều request đồng thời
Khả năng mở rộng	Không hỗ trợ scale, chỉ cho local test	Hỗ trợ scale ngang (nhiều worker, nhiều instance)
Bảo mật	Không an toàn để public lên internet	Dùng ổn định trong **production**, thường kết hợp **Nginx**
Cách chạy	`python app.py` → `app.run()`	`gunicorn app:app --workers 4 --bind 0.0.0.0:8000`
Môi trường sử dụng	Development (debug, test nhanh)	Production (triển khai thật)

Restful API - Flask vs. FastAPI

Tiêu chí	Flask	FastAPI
Ra đời	2010	2018
Tốc độ	Trung bình	Rất nhanh (async)
Kiểu dữ liệu	Không có kiểm tra type	Hỗ trợ type hints (Pydantic)
Tự động docs (Swagger)	Không có sẵn	Có sẵn `/docs`
Async / Await	Không native	Native support
Validation	Thủ công	Tự động
Cộng đồng	Lớn, lâu đời	Đang phát triển nhanh
Phù hợp với	API nhỏ, truyền thống	API async, microservice, ML serving

Restful API - FastAPI with Uvicorn

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str

@app.post("/users")
def create_user(user: User):
    return {"message": f"Hello {user.name}!"}
```

Run: uvicorn simple_api:app

Docs tự động: http://127.0.0.1:8000/**docs**

Restful API - FastAPI with Pydantic

Thư viện dùng để **định nghĩa schema dữ liệu** bằng Python class.

Tự động validate và parse dữ liệu từ request (JSON → Python object).

FastAPI **tích hợp sẵn Pydantic** để:

- Xác thực đầu vào (`request body`, `query params`)
  - Có thể gán giá trị mặc định
  - Parsing thông minh (“123” -> int(123) nếu hợp lệ)
- Tự động trả lỗi chi tiết, rõ ràng cho client
- Sinh tài liệu API tự động (OpenAPI / Swagger) trên **/docs**

Restful API - WSGI

WSGI (**Web Server Gateway Interface**): Chuẩn giao tiếp giữa **web server** (như Gunicorn, uWSGI) và **ứng dụng Python (Flask, Django)**

- **Cơ chế**: *đồng bộ (synchronous)*
  - Xử lý **một request tại một thời điểm** trên mỗi worker
  - Mỗi request đợi hoàn thành rồi mới nhận request mới

- **Phù hợp cho**:
  - Web app truyền thống (CRUD, HTML, API đơn giản)
  - Framework: **Flask, Django (trước 3.0)**
  - Server: **Gunicorn, uWSGI, Waitress**

Restful API - WSGI with Gunicorn

Mỗi worker process hoặc thread chỉ xử lý 1 request tại 1 thời điểm. Trong khi worker đó đang bận (ví dụ chờ query DB, chờ API bên ngoài),

→ các request khác phải chờ trong hàng đợi.

Để **tăng throughput**, bạn có 2 lựa chọn:

- **Tăng số process**: gunicorn -w 4 simple_api:app
  - Xử lý được 4 request song song
- **Dùng thread worker**: gunicorn -w 2 --threads 4 simple_api:app
  - Có thể xử lý 8 request cùng lúc

Nhưng thực ra nó chỉ giả song song vì mỗi thread vẫn chờ I/O; Python bị giới hạn bởi GIL (Global Interpreter Lock)

→ nên hiệu quả thực tế tăng không nhiều với các tác vụ nặng CPU

Restful API - WSGI

Hạn chế:

Vấn đề	Giải thích
Blocking I/O	Nếu 1 request gọi API ngoài mất 2s, thread đó "chết đứng" 2s
Tốn tài nguyên	Mỗi process/thread chiếm RAM, context switching
Không xử lý WebSocket	Vì WSGI không hỗ trợ kết nối lâu dài (persistent)

Restful API - ASGI

ASGI (**Asynchronous Server Gateway Interface**): chuẩn mới thay thế WSGI, hỗ trợ **I/O bất đồng bộ (async)**

- Cơ chế: asynchronous & concurrent
  - Có thể xử lý **nhiều request cùng lúc**
  - Tận dụng **asyncio**, không cần nhiều thread

- Phù hợp cho:
  - Ứng dụng **real-time, websocket, API tốc độ cao**
  - Framework: **FastAPI, Django 3+, Starlette**
  - Server: **Uvicorn, Hypercorn, Daphne**

Restful API - WSGI vs. ASGI

Đặc điểm	WSGI	ASGI
Cơ chế	Đồng bộ	Bất đồng bộ
Hiệu năng	Tốt với app nhỏ	Cao khi nhiều request I/O
Hỗ trợ WebSocket	❌ Không	✅ Có
Framework điển hình	Flask, Django (cũ)	FastAPI, Starlette, Django 3+
Server phổ biến	Gunicorn, Waitress	Uvicorn, Daphne, Hypercorn

Restful API - Gunicorn & Uvicorn

Tiêu chí	Gunicorn	Uvicorn
Loại server	**WSGI HTTP Server** (chuẩn cũ, sync)	**ASGI Server** (chuẩn mới, async)
Dùng cho	Flask, Django (WSGI apps)	FastAPI, Starlette, Django ASGI
Cơ chế	Multi-process, blocking (sync)	Async non-blocking I/O
Hỗ trợ Async	❌ Không hỗ trợ async	✅ Native async/await
Performance	Tốt cho app sync (CPU-bound)	Tối ưu I/O-bound, nhiều kết nối đồng thời
Dùng trong Production	✅ Có (ổn định, lâu đời)	✅ Có (nhẹ, tốc độ cao)
Chạy trực tiếp	`gunicorn app:app`	`uvicorn main:app`
Tích hợp load balancing	✅ Có sẵn (multi-worker)	❌ Phải tự chạy nhiều instance hoặc dùng Gunicorn wrapper
Chuẩn giao tiếp	WSGI	ASGI

Restful API - Gunicorn & Uvicorn

Gunicorn có thể **chạy Uvicorn làm worker** để vừa có:

- Hiệu năng **async** (từ **Uvicorn**)
- Quản lý **worker**, **reload**, **logging** (từ **Gunicorn**)

```
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000
```

Restful API - FastAPI async & await for I/O bound

FastAPI (async, non-blocking):

```python
@app.get("/sleep")
async def sleep():
    await asyncio.sleep(1)
    return {"msg": "done"}
```

Flask (blocking):

```python
@app.route("/sleep")
def sleep():
    time.sleep(1)
    return "done"
```

Encoding vs. Encryption vs. Hashing

Thuật ngữ	Mục đích	Có thể giải mã lại?	Ví dụ
Encoding	Truyền dữ liệu an toàn, tương thích hệ thống	✅ Có	Base64, URL encoding
Encryption	Bảo mật dữ liệu khỏi truy cập trái phép	✅ Có (nếu có key)	AES, RSA
Hashing	Kiểm tra tính toàn vẹn / xác thực	❌ Không	SHA256, bcrypt

Tóm lại:

- **Encoding:** reversible
- **Encryption:** irreversible without key
- **Hashing:** one-way

Encoding

Chuyển dữ liệu sang dạng khác để dễ lưu trữ hoặc truyền đi, không phải để bảo mật.

```
import base64

text = "Hello World"
encoded = base64.b64encode(text.encode())
decoded = base64.b64decode(encoded)
print(encoded)  # SGVsbG8gV29ybGQ=
```

Ứng dụng:

- Email (MIME)
- JWT header & payload
- URL-safe transmission

Encoding - Email

Vấn đề:

- Email theo chuẩn cũ (SMTP) chỉ cho phép ký tự ASCII (0–127).
- Nhưng email thực tế có thể chứa **ảnh, file, text tiếng Việt, Unicode, HTML...**

Giải pháp:

- MIME (Multipurpose Internet Mail Extensions) định nghĩa cách **encode dữ liệu nhị phân hoặc non-ASCII** thành ASCII an toàn.
- Dạng phổ biến: **Base64 encoding** cho file đính kèm (attachments).

Ví dụ:

```
Content-Transfer-Encoding: base64
```

=> File PDF, hình ảnh… được encode base64 để gửi qua email mà không bị hỏng dữ liệu

Encoding - JWT

Vấn đề:

- JWT (JSON Web Token) là chuỗi gồm 3 phần: `header.payload.signature`
- Mỗi phần là **JSON**, nhưng cần truyền qua **HTTP header hoặc URL**.
- Một số ký tự trong JSON hoặc Base64 có thể **không an toàn trong URL** (`+`, `/`, `=`).

Giải pháp:

- JWT dùng **Base64URL encoding** (một biến thể của Base64):
  - `+` → `-`
  - `/` → `_`
  - Bỏ `=`
- Điều này giúp token **có thể truyền an toàn trong URL hoặc header** mà không cần encode thêm lần nữa.

Encoding - JWT

Ví dụ:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

→ Base64URL → `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9`

Encoding - URL-safe transmission

Vấn đề:

- URL chỉ cho phép một số ký tự hợp lệ (theo RFC 3986).
- Ký tự như `+`, `/`, `=`, `?`, `&`… có ý nghĩa đặc biệt trong URL.
- Nếu gửi dữ liệu nhị phân hoặc text chứa các ký tự này, **URL sẽ bị lỗi hoặc cắt sai**.

Giải pháp:

- Dùng **URL encoding** (`%xx` dạng hex) hoặc **Base64URL encoding**.
- Như vậy, dữ liệu (token, binary, JSON...) có thể nằm an toàn trong URL query hoặc path.

Ví dụ:

```
https://example.com/callback?token=eyJhbGciOiJIUzI1NiIs...
```

→ An toàn, không bị cắt bởi dấu &, ? hay =

Encoding

Môi trường	Encoding dùng	Lý do chính	Loại dữ liệu
Email (MIME)	Base64	SMTP chỉ hỗ trợ ASCII	File, binary, text Unicode
JWT Header & Payload	Base64URL	Truyền trong HTTP header / URL	JSON
URL transmission	URL encoding / Base64URL	URL không chấp nhận ký tự đặc biệt	Binary, token, text

Encryption

Chuyển dữ liệu sang dạng không đọc được nếu không có **key**

Phân loại:

- **Symmetric** (đối xứng): 1 key duy nhất (AES, DES)
- **Asymmetric** (bất đối xứng): public/private key (RSA)

Ứng dụng:

- HTTPS / TLS
- Lưu trữ dữ liệu nhạy cảm
- Truyền file an toàn

Symmetric Encryption

Symmetric Encryption:

- **Dùng cùng một key** cho cả mã hóa và giải mã.
- Nhanh, hiệu quả → dùng trong thực tế để mã hóa dữ liệu lớn.
- Cần **bảo mật key tuyệt đối** (nếu bị lộ, dữ liệu bị giải mã được).

Ví dụ:

- AES (Advanced Encryption Standard)
- DES (Data Encryption Standard)
- ChaCha20

Ứng dụng:

- Mã hóa ổ cứng (BitLocker)
- File lưu trữ bảo mật
- Giao thức HTTPS (sau khi handshake RSA, dùng AES để mã hóa dữ liệu)

Symmetric Encryption

- Plaintext → [Encrypt with **Key K**] → Ciphertext
- Ciphertext → [Decrypt with **same Key K**] → Plaintext

```python
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

# --- ENCRYPT ---
key = get_random_bytes(16)  # 128-bit key
cipher = AES.new(key, AES.MODE_EAX)

plaintext = b"My secret m
ciphertext, tag = cipher.encrypt_and_digest(plaintext)

# Lưu lại nonce, tag, ciphertext để sau này decrypt
nonce = cipher.nonce

print("Ciphertext:", ciphertext.hex()) # b8e38182d29e402e8adfae0c6462192c01
print("Tag:", tag.hex()) # e4420b38a47d894d821b685092039310 (dùng để xác thực (authentication tag) đảm bảo dữ liệu không bị sửa đổi)
print("Nonce:", nonce.hex()) # f74de720e9f68e3ecaf32643d81c6cfd (giá trị ngẫu nhiên mỗi lần mã hóa, giúp tăng tính bảo mật)

# --- DECRYPT ---
cipher_dec = AES.new(key, AES.MODE_EAX, nonce=nonce)
plaintext_dec = cipher_dec.decrypt_and_verify(ciphertext, tag) # (vừa giải mã vừa kiểm tra tính toàn vẹn dữ liệu)

print("Decrypted:", plaintext_dec.decode()) # My secret message
```

(function) encrypt_and_digest: Any

Asymmetric Encryption

- Dùng **2 key khác nhau**:
  - **Public Key**: dùng để **mã hóa**
  - **Private Key**: dùng để **giải mã**
- Nếu ai có public key, họ có thể gửi dữ liệu mã hóa cho bạn → chỉ bạn (với private key) mới đọc được.

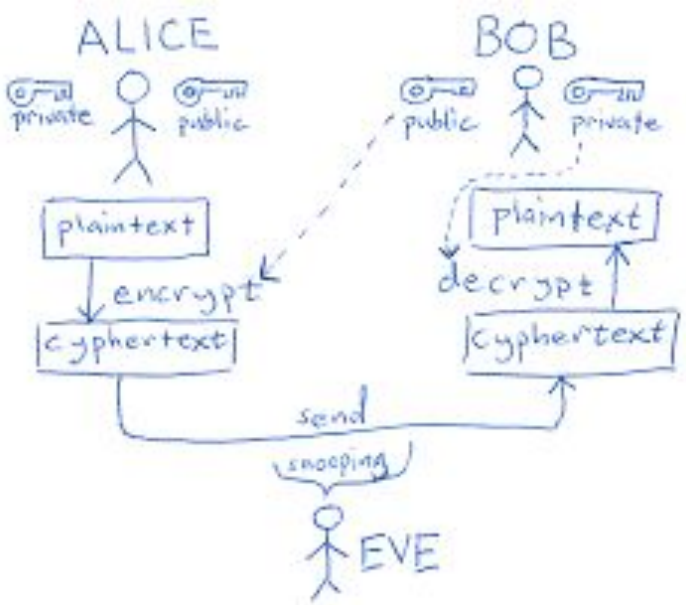
Ví dụ:

- RSA (Rivest–Shamir–Adleman)
- ECC (Elliptic Curve Cryptography)

Ứng dụng:

- Giao thức HTTPS (TLS handshake)
- SSH key
- JWT signing
- Blockchain wallet

Asymmetric Encryption



Tính chất:

- Không cần chia sẻ private key qua mạng
- An toàn hơn symmetric khi truyền key

Encryption

Vấn đề: RSA an toàn nhưng chậm, **AES** nhanh nhưng cần chia sẻ key.

Giải pháp: Kết hợp cả hai:

1. Dùng **RSA** để mã hóa **AES key**
2. Dùng **AES** để mã hóa dữ liệu thực tế

Ứng dụng:

- HTTPS handshake
- Signal, WhatsApp
- PGP email encryption

Encryption

RSA an toàn nhưng rất chậm, không phù hợp để mã hóa dữ liệu lớn.

Do đó HTTPS **chỉ dùng RSA để trao đổi key, còn dùng AES để mã hóa dữ liệu thực tế.**

Quy trình rút gọn:

1. **Client** gửi yêu cầu handshake đến **Server**
2. **Server** gửi **Public Key (RSA/ECC)** của nó cho **Client**
3. **Client**:
    a. Tạo ra một **session key (AES key)** ngẫu nhiên
    b. Mã hóa **session key** này bằng **RSA public key**
    c. Gửi ngược lại cho **server**
4. **Server**:
    a. Giải mã **session key** bằng **RSA private key**
    b. Cả hai bên giờ cùng có chung **AES key**
5. Từ đây trở đi: Mọi dữ liệu thực tế được mã hóa bằng **AES (Symmetric)**

Encryption

RSA rất chậm

- **RSA** dựa trên toán học số lớn (modular exponentiation), cần thao tác với số có 2048 hoặc 4096 bit.
- Trong khi **AES** chỉ dùng phép XOR và S-box lookup, nhanh **gấp hàng trăm đến hàng nghìn lần**.

RSA có thể mã hóa vài KB/s, còn AES hàng trăm MB/s.

RSA không xử lý được dữ liệu lớn

- RSA chỉ mã hóa được tối đa bằng độ dài key (ví dụ RSA-2048 chỉ ~245 byte dữ liệu).
- Không thể dùng RSA trực tiếp cho file, ảnh, hay request HTTPS.

Encryption

AES Rất nhanh & hiệu quả

- Dễ thực thi bằng phần cứng (CPU có lệnh AES-NI).
- Tốc độ cao, mã hóa dữ liệu liên tục trong kết nối HTTPS.

Chỉ cần trao đổi key 1 lần

- Khi handshake xong, hai bên có chung AES key — dùng trong suốt session.

Kết hợp được tính an toàn của RSA

- **RSA** bảo vệ **key** (**confidentiality**)
- **AES** bảo vệ **dữ liệu** (**efficiency**)

Encryption - HTTPS

Thuật toán	Ưu điểm	Nhược điểm	Dùng trong HTTPS
RSA	Bảo mật key, không cần chia sẻ secret	Rất chậm, không dùng cho dữ liệu lớn	Handshake
AES	Rất nhanh, an toàn	Cần chia sẻ key trước	Truyền dữ liệu sau khi handshake

Encryption

Ứng dụng	Công nghệ mã hóa
HTTPS	RSA + AES
Wi-Fi (WPA2)	AES
Ứng dụng ngân hàng	AES, RSA
Blockchain wallet	ECC
Email bảo mật (PGP)	RSA

Hashing

Tạo ra “vân tay” duy nhất từ dữ liệu đầu vào → không thể đảo ngược.

```python
import hashlib

msg = "password123"
print(hashlib.sha256(msg.encode()).hexdigest())  # ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
```

Tính chất:

- Không thể đảo ngược
- Cùng input → cùng hash
- Thay đổi 1 ký tự → hash hoàn toàn khác

Ứng dụng:

- Lưu mật khẩu (bcrypt, Argon2)
- Kiểm tra integrity file
- Blockchain

Encoding vs. Encryption vs. Hashing

Đặc điểm	Encoding	Encryption	Hashing
Mục đích	Truyền dữ liệu	Bảo mật	Xác thực
Đảo ngược được	✅ Có	✅ Có (với key)	❌ Không
Có dùng key?	❌	✅	❌
Dễ crack	Dễ	Trung bình – khó	Không thể đảo (nhưng có thể brute-force)
Ví dụ	Base64	AES, RSA	SHA256, bcrypt

Restful API - Security

Bảo vệ API khỏi truy cập trái phép

Thuộc tính	Authentication	Authorization
Mục đích	Xác minh danh tính người dùng	Kiểm soát quyền truy cập tài nguyên
Khi nào	Khi login	Khi gọi API
Kết quả	Nhận token (JWT)	API được phép / từ chối
Ví dụ	`POST /login`	`GET /users` (phải có role=admin)

Authentication trả token, **Authorization** dùng token đó để xác nhận quyền

Restful API - Security Levels

Cấp độ	Chủ đề	Mục tiêu
Level 1	API Key, JWT, HTTPS	Hiểu cơ bản và triển khai
Level 2	OAuth2, RBAC, Rate Limit	Áp dụng trong hệ thống thực
Level 3	Policy Engine, API Gateway	Triển khai ở quy mô doanh nghiệp
Level 4	mTLS, Zero Trust, Vault	Chuẩn bảo mật hiện đại / ngân hàng

Restful API - Security Levels

- **API Key:** đơn giản, gửi kèm trong header

```
Authorization: ApiKey abc123
```

- **Basic Auth:** username:password (base64)
  ít an toàn, dùng tạm cho dev/test

- **Bearer Token (JWT):** token ngắn hạn, ký bằng secret key
  phổ biến nhất cho REST API

- **OAuth2:** dùng cho login qua Google, Facebook, v.v.

Restful API - API Key

Đặc điểm:

- Là một chuỗi bí mật (key) cấp cho từng client.
- Client gửi API key trong request header, ví dụ:

  Authorization: ApiKey abc123

- Server kiểm tra key này có hợp lệ hay không.

Ưu điểm:

- Dễ dùng, đơn giản cho microservice hoặc hệ thống nội bộ.

Nhược điểm:

- Không có thông tin người dùng (ai đang dùng).
- Nếu bị lộ key → ai cũng gọi API được.

Restful API - Basic Auth

Gửi trực tiếp `username:password` (đã mã hóa **Base64**, nhưng KHÔNG mã hóa thật sự):

```
Authorization: Basic dXNlcjpzZWNyZXQ=
```

Khi decode ra lại thấy rõ `user:secret`.

Tại sao ít an toàn?

- Vì:
  - **Base64** chỉ là **encoding**, **không phải encryption**.
  - Nếu không dùng HTTPS, attacker có thể **nghe lén (sniff)** và thấy rõ username/password.
  - Dễ bị **replay attack** (gửi lại request cũ).
- Do đó chỉ nên dùng cho **dev/test**, hoặc khi API nằm sau HTTPS + firewall nội bộ.

Restful API - Bearer Token (JWT)

JWT là chuỗi mã hóa gồm 3 phần:

```
header.payload.signature
```

Ví dụ:

```
Header = { "alg": "HS256", "typ": "JWT" }
Payload = { "sub": "hoa", "role": "admin", "exp": 1739800000 }
Secret = "my_secret"
```

Khi gởi tới server:

```
Authorization: Bearer
<base64(header)>.<base64(payload)>.<Hash with secret key (payload)>
```

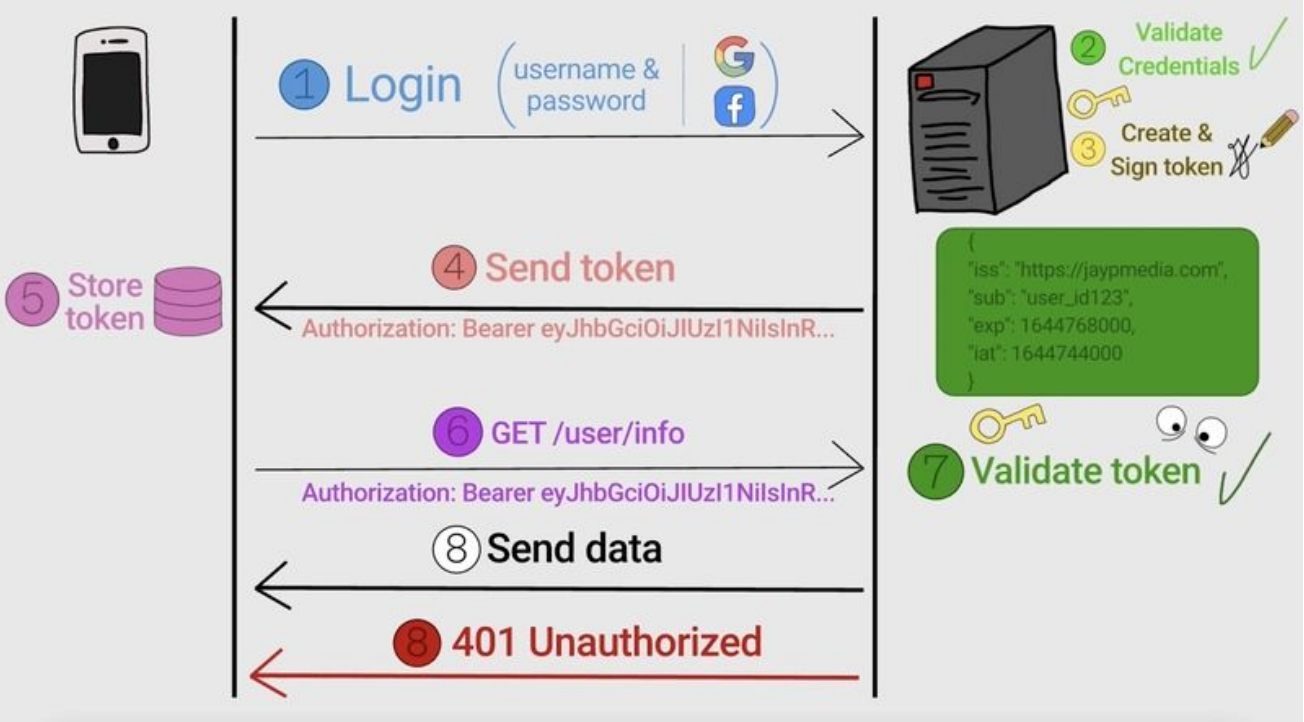
Restful API - Bearer Token (JWT)

```python
import jwt
secret = "my-secret"
token = jwt.encode(payload, secret, algorithm=header['alg'])
print(token)
```

Kết quả:

- eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
- eyJzdWIiOiJob2EiLCJyb2xlIjoiYWRtaW4iLCJleHAiOjE3Mzk4MDAwMDB9.
- oi07FUZpNA5FvxoaHfbVoOx1dwXe9NxbaNkokavjgy4

Restful API - Bearer Token (JWT)



Restful API - Bearer Token (JWT) - Stateless Authentication

- Token **tự mang theo toàn bộ thông tin user và quyền hạn**.
- Server **không cần lưu session** hay truy vấn DB để check login mỗi lần.
- Khi scale nhiều server (load balancer, microservices), chỉ cần **cùng secret key** để verify là đủ → dễ mở rộng.

Restful API - Bearer Token (JWT) - Stateless Authentication

Bảo mật nâng cao trong thực tế:

Cơ chế	Mục đích
HTTPS	chống sniffing token
Expire ngắn (15–30 phút)	giảm rủi ro token bị lộ
Refresh token rotation	cấp lại access token an toàn
HttpOnly cookie	chống XSS đánh cắp token
Revoke list	thu hồi token đã bị nghi ngờ

Restful API - Basic Auth vs. JWT

Basic Auth cũng **stateless**, nhưng **kém bảo mật** và **khó scale hơn**:

(1) JWT không tiết lộ thông tin nhạy cảm

- Basic Auth: nếu hacker **chặn 1 request**, họ có **username/password thật** → chiếm tài khoản.
- JWT: token chỉ là **một chuỗi mã hóa**, không chứa password thật.
  - Phần **payload** (user id, role, exp, …) có thể đọc được (vì base64),
  - nhưng phần **signature** được ký bằng **secret key** (**chỉ server biết**) → hacker không thể giả token.

(2) JWT có thời hạn (expiration)

- Token chỉ sống vài phút hoặc vài giờ → giảm rủi ro bị lộ.
- Basic Auth thì không có hạn dùng: nếu bị leak → nguy cơ lâu dài.

(3) JWT không cần xác thực lại mỗi lần

- Basic Auth: mỗi request đều phải gửi user/pass và check trong DB.
- JWT: server chỉ cần verify chữ ký (rất nhanh, không cần query DB).

  → hiệu năng cao, dễ scale hơn trong hệ thống microservices.

(4) JWT có thể mở rộng bảo mật

- Có thể ký bằng RSA private key, xác minh bằng public key (an toàn hơn khi nhiều service).
- Có thể thêm claim như iss, aud, exp, scope để kiểm soát quyền hạn.

Restful API - Basic Auth vs. JWT

Tiêu chí	Basic Auth	JWT (Bearer Token)
Trạng thái	Stateless	Stateless
Xác thực lại	Phải query DB mỗi lần	Chỉ cần verify chữ ký
Tốc độ	Chậm hơn	Nhanh, phù hợp high-traffic
Bảo mật	Thấp (gửi user/pass trực tiếp)	Cao (token ngắn hạn, ký bằng secret/private key)
Thu hồi / Hết hạn	Không có cơ chế	Có `exp`, có thể revoke
Ứng dụng thực tế	Dev/Test, Internal	Sản phẩm thực tế, REST API, microservice

Restful API - Stateful Auth

I. Giai đoạn login

1. User gửi username/password đến server (`POST /login`).
2. Server xác thực thông tin → nếu đúng, tạo ra một **session** (bản ghi) trên server:

```
session_id = "abc123"
sessions["abc123"] = {
    "user_id": "hoa",
    "role": "admin",
    "expires": "2025-10-18T16:00:00"
}
```

1. Server trả về cookie chứa session_id:

```
Set-Cookie: session_id=abc123; HttpOnly; Secure
```

Restful API - Stateful Auth

II. Giai đoạn truy cập API

Khi client (trình duyệt) gọi API tiếp theo, cookie session_id tự động được gửi kèm:

```
GET /dashboard
Cookie: session_id=abc123
```

III. Server xác thực request

- Server nhận session_id.
- Kiểm tra trong bộ nhớ hoặc database xem session đó còn hợp lệ không.
- Nếu session hợp lệ → lấy user info từ đó → cho phép truy cập.
- Nếu hết hạn hoặc không tồn tại → trả lỗi 401 Unauthorized.

IV. Khi logout

- Server xóa session_id khỏi bộ nhớ/DB.
- Cookie ở client cũng hết hiệu lực → user bị đăng xuất.

Restful API - Stateful Auth

Vì sao gọi là Stateful ?

- Bởi vì server **phải lưu "trạng thái đăng nhập" (session)** của từng user.
- Mỗi request phụ thuộc vào thông tin đã lưu ở server (session state).
- Nếu bạn chạy **nhiều server backend**, cần **chia sẻ session state** qua:
  - Redis, Memcached, hoặc
  - Database (MySQL/Postgres session table)

  → Để tất cả server biết user nào đang đăng nhập.

Restful API - Stateful vs. Stateless Auth

Tiêu chí	Stateful (Session)	Stateless (JWT, Basic, API Key)
Server lưu trạng thái	✅ Có (session store)	❌ Không
Khả năng scale	Khó hơn (phải share session)	Dễ scale (mỗi request tự verify)
Tốc độ verify	Nhanh (truy vấn RAM/Redis)	Nhanh (verify chữ ký)
Đăng xuất (logout)	Dễ — xóa session	Khó hơn — phải revoke token
Bảo mật	An toàn, vì session_id khó đoán	Tùy thuộc cách ký token / secret
Phổ biến ở	Web apps truyền thống	REST API, microservices, mobile apps

Restful API - OAuth2

Ứng dụng bên thứ ba (ví dụ: *Spotify*) muốn truy cập tài khoản Google của người dùng.

Nếu không có OAuth2:

- App phải yêu cầu **username/password** của user → cực kỳ rủi ro.
- User không thể **giới hạn quyền truy cập** (Spotify có thể đọc toàn bộ Gmail).
- Không thể **revoke** dễ dàng.

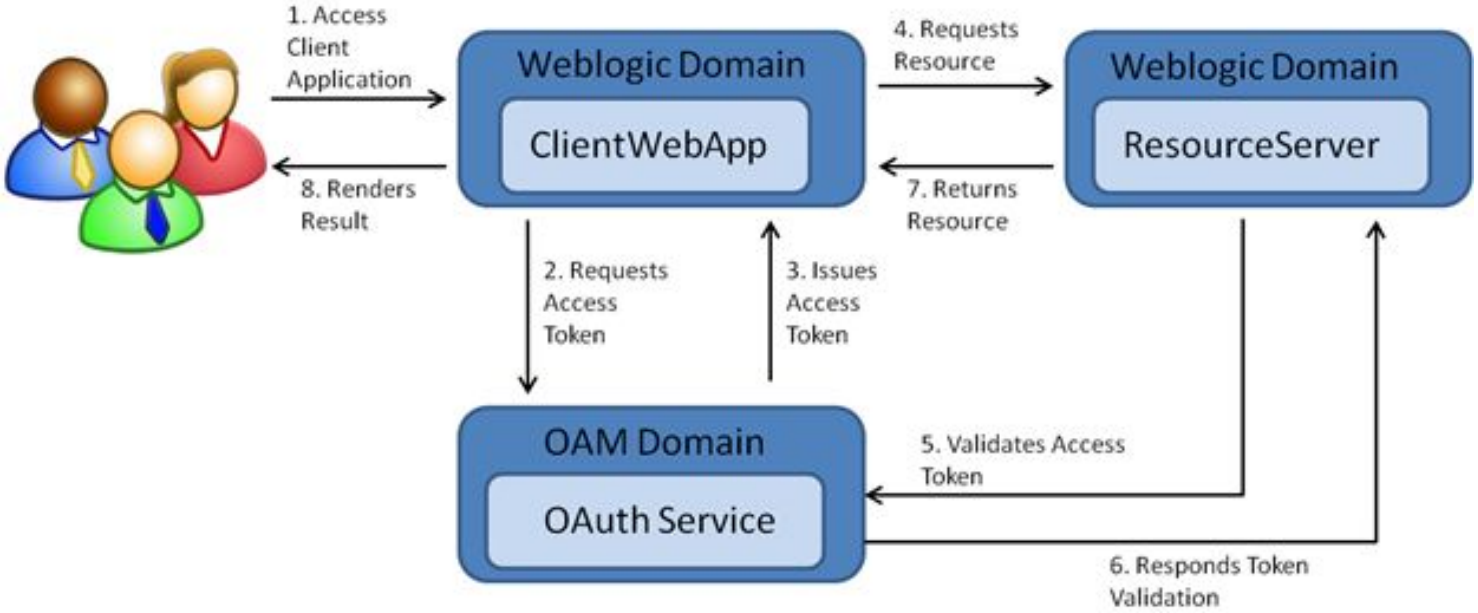
OAuth2 ra đời để giải quyết: "ủy quyền *một phần quyền truy cập*" mà **không cần chia sẻ mật khẩu.**

Restful API - OAuth2

Các vai trò trong OAuth2:

Thành phần	Mô tả
Resource Owner	Người dùng (user)
Client	App muốn truy cập dữ liệu
Resource Server	Nơi chứa dữ liệu (API server)
Authorization Server	Xác thực user, cấp token

Restful API - OAuth2



- **Auth Server ký (sign)** token bằng **private key** (RS256).
- **Resource Server** sau này **verify** token bằng **public key**.

Restful API - OAuth2

Authorization Server phải công khai public key thông qua endpoint chuẩn gọi là:

```
GET /.well-known/jwks.json
```

Ví dụ:

```
{
  "keys": [
    {
      "kty": "RSA",
      "use": "sig",
      "kid": "abc123",
      "alg": "RS256",
      "n": "base64-modulus",
      "e": "AQAB"
    }
  ]
}
```

Resource Server:

1. Khi khởi động hoặc định kỳ (cache vài phút/giờ),
2. Gọi đến endpoint này,
3. Lưu lại public key vào memory hoặc cache,
4. Khi có JWT mới tới, đọc `kid` trong header,
5. Dùng đúng public key tương ứng để verify.

RPC

RPC = **Remote Procedure Call** (“Gọi hàm ở service khác như gọi hàm local.”)

```
Client App
  └──> client_stub (auto-generated)
       └── network (TCP/HTTP2)
            └── server_stub → handler → logic
```

RPC abstract hóa phần networking. Dev chỉ viết hàm, còn serialization, transport, reconnect… do framework lo

RPC vs. Restful API

Tiêu chí	REST	RPC
Triết lý	Resource-based	Function-based
Format	JSON	Binary (Protobuf, Thrift, Avro)
Tốc độ	Chậm hơn	Nhanh hơn
Streaming	Giới hạn	Native support
Debug / Tooling	Dễ dùng	Cần codegen
Dùng cho	Public APIs	Internal microservices
Example	`GET /users/123`	`UserService.GetUser(123)`

RPC: Thrift & gRPC

	Thrift	gRPC
Origin	Facebook	Google
Protocol	Binary / Compact	Protobuf
Transport	TCP (custom)	HTTP/2
Multiplexing	❌	✅
Streaming	⚙️ thủ công	✅ native
Pooling	Manual	Built-in
Typical use	legacy, infra service	modern microservice

Connection Pooling

Tại sao cần connection pool ?

Mỗi RPC call cần:

- TCP handshake (3-way)
- SSL handshake (nếu có)
- Round-trip latency

Mở/đóng TCP liên tục = tốn CPU và thời gian.

Giải pháp: Giữ connection mở → tái sử dụng qua pool.

Connection Pooling

Cơ chế:

- Duy trì sẵn N connections đến server.
- Mỗi request lấy 1 connection rảnh.
- Sau khi xong → trả lại pool.
- Nếu pool đầy → đợi hoặc mở thêm.

Giống connection pool của database (Postgres/MySQL)

Connection Pooling

```
[Client]
 ├── Pool
 │     ├── conn1 → busy
 │     ├── conn2 → busy
 │     └── conn3 → idle
 └──> lấy conn3 → gửi RPC
```

Nếu 10 threads cùng gửi request mà chỉ có 2 connection, sẽ bị queue

- max_pool_size
- min_idle
- max_idle_time
- connect_timeout
- max_lifetime
- retry_policy

(*) Cấu hình sai có thể dẫn đến connection leak hoặc spike latency

Thrift Pooling vs gRPC Pooling

	Thrift	gRPC
Pooling	Manual (phải tự code)	Built-in Channel Pool
Multiplexing	❌ 1 conn = 1 request	✅ 1 conn = nhiều stream
Reconnect	Manual retry	Auto reconnect
Timeout	Manual	Built-in deadline per call
Use	Suitable cho low-level system	Suitable cho microservices

Transport Layer

Là lớp truyền tải dữ liệu giữa client và server RPC.

Thrift:

- `TSocket` – raw TCP
- `TBufferedTransport` – có buffer
- `TFramedTransport` – gói frame cố định

gRPC: dùng HTTP/2 với stream và flow control.

Transport ảnh hưởng trực tiếp tới throughput & latency.

gRPC Multiplexing

Cho phép **nhiều RPC chạy song song trên 1 TCP connection.**

- REST (HTTP/1.1): mỗi request một connection.
- **Thrift** (TCP raw): 1 conn → 1 request.
- **gRPC** (HTTP/2): nhiều stream ID khác nhau chạy đồng thời.

```
HTTP/1.1
 conn1: req1 → resp1
 conn2: req2 → resp2

HTTP/2
 conn1:
   stream1 → resp1
   stream2 → resp2
   stream3 → resp3
```

HTTP/2 multiplexing giúp giảm socket và latency khi có nhiều parallel call

Multiplexing vs. Connection Pooling

Tiêu chí	Connection Pooling	Multiplexing
Mục đích	Giảm chi phí tạo kết nối	Giảm số lượng kết nối, tăng concurrency
Độ song song	Mỗi conn 1 request tại 1 thời điểm	1 conn nhiều request đồng thời
Cần hỗ trợ protocol đặc biệt	Không (HTTP/1.1, JDBC pool...)	Có (HTTP/2, gRPC, Thrift mux)
Chi phí setup	Thấp hơn so với tạo conn mới	Cao hơn, nhưng hiệu quả hơn về lâu dài

gRPC Streaming

- **gRPC** hỗ trợ streaming native
- **Thrift** không có streaming thực thụ trong thiết kế gốc, chỉ có *fake streaming* (chunking / long polling) nếu tự implement thêm.

4 loại gRPC Streaming

Loại	Mô tả	Use case
Unary	1 → 1	CRUD
Server-streaming	1 → nhiều	log, report, progress
Client-streaming	nhiều → 1	upload file, metrics
Bidirectional	nhiều ↔ nhiều	chat, telemetry, LLM inference

gRPC Streaming - Server Streaming

```
Client: request job_result(id)
Server: stream progress
  → 20%
  → 50%
  → 100% done
```

Giúp client nhận dữ liệu dần thay vì chờ 1 response lớn

gRPC Streaming - Client Streaming

```
Client: stream chunks
 → chunk1
 → chunk2
 → chunk3
Server: process & return summary
```

Upload file lớn, hoặc stream sensor data theo batch

gRPC Streaming - Bidirectional Streaming

```
Client ↔ Server
 → msg1    ← ack1
 → msg2    ← ack2
 → msg3    ← ack3
```

Thường dùng cho chat, real-time analytics, fraud detection, voice recognition

gRPC Streaming

Ưu điểm:

- Low latency, không cần reconnect
- Giảm overhead handshake
- Multiplex stream song song
- Type-safe, binary, có schema
- Không hoạt động trực tiếp trên browser (phải qua gRPC-Web)

gRPC Streaming vs. WebSocket

	WebSocket	gRPC bidirectional
Layer	TCP (HTTP/1.1 upgrade)	HTTP/2
Multiplexing	❌ 1 channel	✅ nhiều stream
Schema	JSON tự do	Protobuf strict
Lifecycle	persistent socket	per-RPC stream
Use	frontend realtime	backend realtime

Restful API vs. Thrift vs. gRPC

	REST	Thrift	gRPC
Serialization	JSON	Binary	Protobuf
Transport	HTTP/1.1	TCP	HTTP/2
Multiplexing	❌	❌	✅
Pooling	HTTP Keep-Alive	Manual	Built-in
Streaming	❌	⚙️	✅
Latency	cao	TB	thấp
Dev Effort	thấp	cao	TB

High Performance RPC

4 yếu tố quyết định hiệu năng RPC:

1. **Connection Pooling** → giảm overhead TCP
2. **Transport** → TCP vs HTTP/2
3. **Multiplexing** → tận dụng 1 connection cho nhiều stream
4. **Streaming** → tối ưu real-time & long-running ops

Event-driven Architecture

Recap:

- REST → HTTP-based, stateless
- RPC / gRPC / Thrift → synchronous request/response
- Hạn chế: không phù hợp với các task chạy lâu, real-time, hoặc callback event

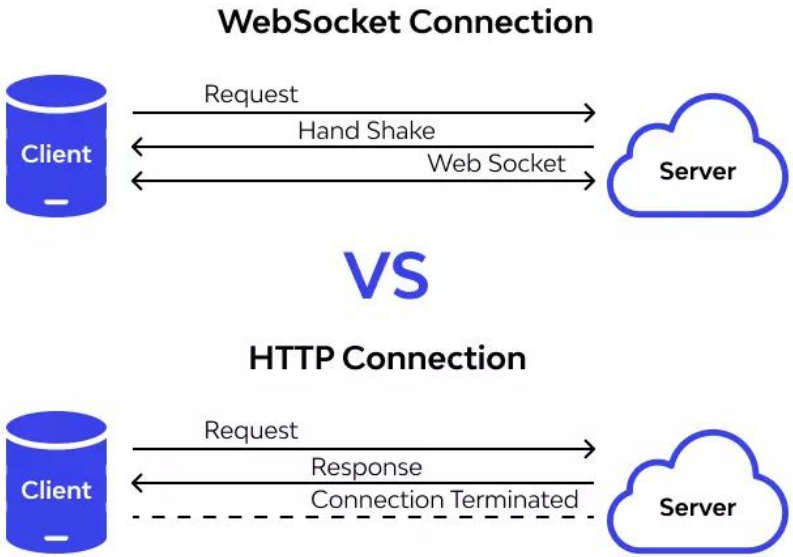
Thay vì client gọi → chờ trả lời, hệ thống sẽ:

- Gửi event (trigger)
- Hệ thống xử lý async
- Push ngược kết quả qua Webhook / WebSocket

REST/RPC vs. Websocket vs. Webhook vs. Celery

Cơ chế	Communication	Direction	Persistent	Use-case
REST/RPC	Request-Response	Client→Server	❌	Sync API
WebSocket	Bi-directional Stream	Both	✅	Real-time
Webhook	Event Callback	Server→Client	❌	Async event
Celery	Background Queue	Internal	✅	Heavy async job

Websocket



- Là giao thức full-duplex dựa trên TCP
- Giữ connection sau khi “upgrade” từ HTTP
- Dùng trong: chat app, notification, live dashboard

HTTP	WebSocket
Request/Response	Persistent connection
Client-initiated	Both sides can send
Stateless	Stateful
High overhead	Low latency

Websocket - How long the connection is held ?

Lý thuyết: Kết nối WebSocket **được giữ mãi** cho đến khi **một trong hai bên** đóng nó, hoặc **mạng bị gián đoạn**. Không có giới hạn thời gian mặc định trong giao thức WebSocket.

Trong thực tế:

Thành phần	Có thể ngắt sau	Lý do
Client (browser)	Tùy hệ điều hành và trình duyệt	Khi tab bị đóng, ngủ máy, mất mạng, timeout nội bộ
Server (FastAPI/Uvicorn)	Không timeout mặc định	Nhưng nếu bạn dùng reverse proxy (nginx) hoặc load balancer, sẽ có giới hạn
Reverse proxy (nginx, Cloudflare, AWS ALB, etc.)	30s → 1h (tùy cấu hình)	Để tránh giữ TCP lâu gây tốn tài nguyên
Firewall / NAT	5–10 phút nếu idle (không có dữ liệu)	Hệ thống mạng tự đóng kết nối "nhàn rỗi"

Websocket - Why it is more efficient than Pooling ?

Giả sử có **10.000 client**, mỗi client polling API 1 giây/lần → 10.000 req/s = **tải CPU và I/O khổng lồ**

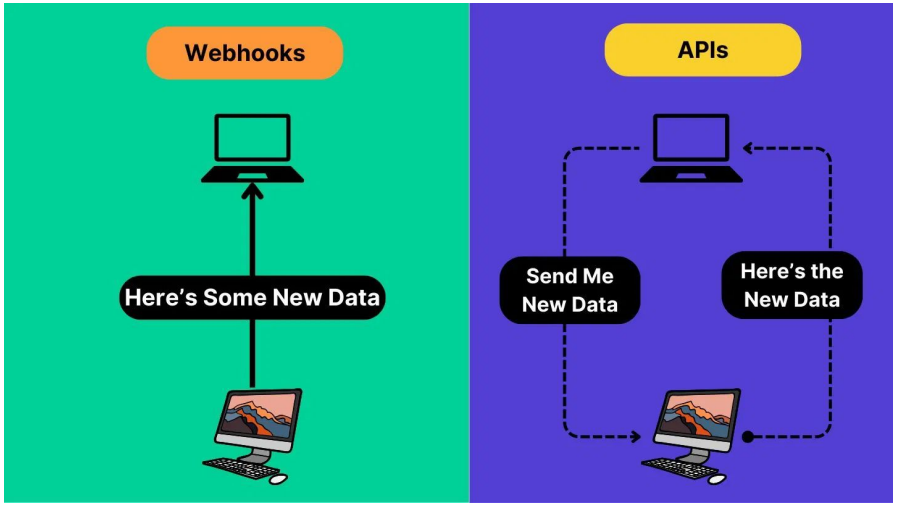
Trong khi WebSocket:

- Chỉ mở **10.000 TCP connection** một lần
- Gửi dữ liệu **chỉ khi có sự kiện**
- Ít overhead HTTP headers → tiết kiệm băng thông

Vì vậy **WebSocket** hiệu quả hơn nhiều cho realtime messaging, dashboard, hay notification.

Thành phần	Tốn bao nhiêu	Ghi chú
RAM	~50–100 KB / connection	chủ yếu cho buffer & coroutine
CPU	rất thấp nếu idle	chỉ tăng khi có traffic
Socket handle	1 / connection	giới hạn OS ~65k / process
Scale tốt nhất	10k–50k per node	hơn nữa thì dùng horizontal scaling

Webhook



Webhook:

- Là một HTTP POST callback khi có sự kiện xảy ra
- "Server pushes data → Client endpoint"

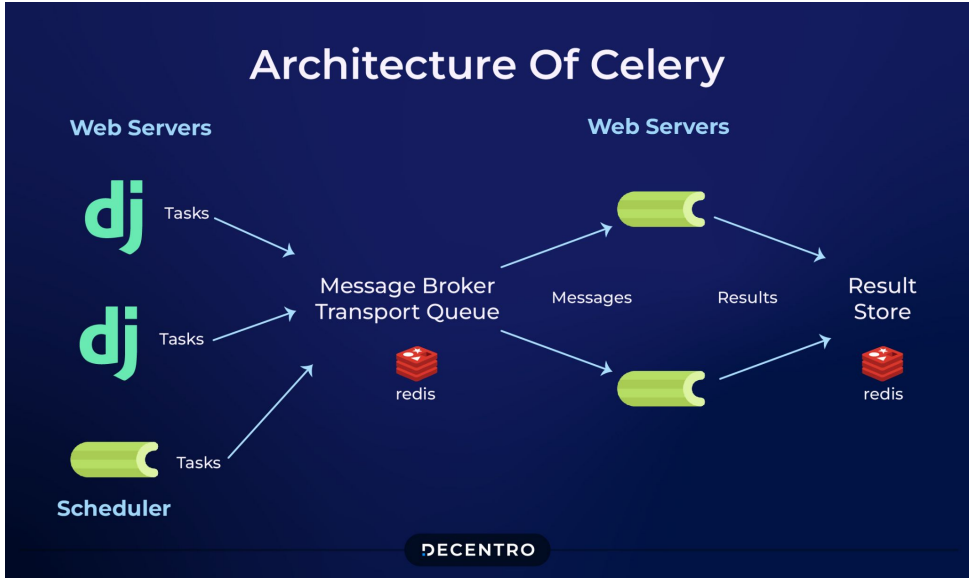
Cách hoạt động:

1. **Client** đăng ký webhook URL
2. **Server** lưu URL
3. Khi có event → **server** gửi POST payload

Use-cases:

- **Payment system** → gửi webhook khi thanh toán xong
- **ML training service** → gửi webhook khi model done
- **GitHub / Slack** → trigger automation

Celery tasks



Khi request quá nặng (VD: train model, gửi email hàng loạt...)

→ server phải đợi xong mới trả response

→ client timeout, user chờ lâu, tốn CPU.

Giải pháp: đưa task sang background worker xử lý sau

→ web server chỉ cần nhận yêu cầu, xếp hàng task.

Cách hoạt động:

- FastAPI nhận request
- Đẩy task Celery, dùng Redis làm Transport Queue
- Worker chạy xong → gọi webhook hoặc push WebSocket message

Celery tasks vs. Thread vs. AsyncIO

Cách	Ưu điểm	Nhược điểm
ThreadPool	Dễ implement	Không scale ra nhiều host
AsyncIO	Gọn nhẹ	Không dùng cho CPU-heavy
Celery	Scale multi-host	Setup phức tạp hơn